

AWS S3

Why use Amazon S3? — Full list (every point included)

When comparing **EBS**, **EFS**, and **S3**, here's a complete, point-by-point explanation of why we use **S3** for object storage.

Key reasons

1. Limited storage in EBS.

EBS volumes are fixed-size — you pay for the full volume even if you don't use it.

2. EFS is expensive.

Example (approx):

- EFS → 1 GB ≈ ₹30
- S3 → 1 GB ≈ ₹2

3. Hard to manage / track multiple EBS volumes.

When you store many files across multiple EBS volumes it becomes hard to track. S3 centralizes objects in a bucket and is easier to manage.

4. EFS doesn't provide direct HTTP URLs like S3.

S3 gives object endpoints / URLs you can open in a browser.

5. Different primary use-cases.

- EFS → application data (shared filesystem, source code)
- S3 → object storage (files, images, backups, media, static websites)

6. Unlimited storage.

S3 scales automatically — you pay only for what you use.

7. Common/shared storage.

Multiple applications or users can access the same bucket.

8. **Redundant storage across AZs.**

Objects are stored redundantly across 3 or more Availability Zones.

9. **Very high availability.**

Availability is very high (e.g., ~99.999% or better depending on class).

10. **Very high durability.**

“11 nines” durability — 99.999999999% — designed to prevent data corruption.

11. **Data in S3 is encrypted by default.**

This helps keep data secure at rest.

12. **Fine-grained permissions.**

Unlike EBS folders, S3 allows policies at object-level (per file) for precise access control.

13. **Protection against accidental deletes.**

Use Versioning and Object Lock (or bucket policies) to prevent accidental deletion.

Short summary

- Unlimited storage
 - Common shared storage
 - 11 nines → durability
 - Very high availability (~99.999%)
 - Low cost — pay only for what you use
-

S3 object basics

- **Bucket name must be globally unique.**
- S3 stores **objects**. Each object contains:
 1. File
 2. Metadata
 3. Key → the “path” of the file inside the bucket

- **S3 does not have real folders.**

Folders are a UI concept created from object keys.

Bucket endpoint example (shows folder-like path)

```
http://<bucket_name>.s3.<region>.amazonaws.com/images/abc.jpg
```

`/images/abc.jpg` looks like a folder path but it is actually part of the object key.

Default access & public access

- **By default, S3 buckets are private.**
 - To access files publicly you must set bucket or object permissions.
 - You can make the **bucket public** and then allow public access only to selected objects via policies.
-

ACL disabled vs ACL enabled

1. ACL disabled (recommended)

- Use bucket policies / IAM for fine-grained control.
- Everything is private by default.
- Allows to define permission at deep level.
- You can allow or deny specific actions (upload, read, delete).

2. ACL enabled

- Simple Read/Write permissions via ACLs.
 - Less flexible than policies. Good for simple use-cases.
-

Creating ACL Enabled Bucket (Practical)

1. Go to AWS Console → S3.

Write bucket name → select **ACL enabled**.